StockSage AI

Agentic Stock Analysis Platform

Technical Specification & Build Blueprint · v1.0

Prepared for	
Platform	
Stack	
LLM	
Backend	
Status	

Table of Contents

1	Executive Summary	3
2	No-GPU LLM Strategy	3
3	User Interaction Model	4
4	Backend Communication Design	5
5	Complete Free Tech Stack	6
6	Data Sources & Catalog	7
7	Data Ingestion Pipelines	8
8	Agent Design	11
9	Frontend Architecture	16
10	Project File Structure	17
11	Environment Setup	18
12	Implementation Roadmap	19
13	Appendix: Key Code Snippets	20

1. Executive Summary

StockSage AI is a production-grade, cross-platform application that combines Agentic AI, Machine Learning, Technical Analysis, Fundamental Analysis, and real-time News Sentiment to deliver near-future stock predictions for individual stocks and user portfolios. The platform is built entirely on free and open-source tools, with no GPU required for operation.

What it does	Analyses any NSE/BSE stock using 3 specialised AI agents (Fundamental, Technical, Sentiment) orchestrated by LangGraph, producing a prediction with direction, target range, and confidence score.
Who uses it	Retail investors in India managing personal portfolios; anyone tracking NSE/BSE/global stocks.
Key differentiator	Live reasoning trace streamed to the user as agents work — not a black box. User sees each agent's thinking step-by-step via WebSocket.
Cost to build v1	Rs 0 / month during development. Groq free tier + Google Gemini free tier + yfinance + open-source libraries. No GPU required at any stage.

2. No-GPU Strategy — How We Run Without Ollama

Ollama requires a local GPU (ideally 8GB+ VRAM) to run large language models at usable speed. Since no local GPU is available, the architecture fully eliminates Ollama and replaces it with two complementary cloud-based free LLM tiers. This is actually a better choice for a deployed production app — cloud APIs are faster, always available, and require zero hardware maintenance.

2.1 Primary LLM: Groq API (Free Tier)

Groq is a hardware-accelerated inference provider offering genuinely free API access to Llama 4 Scout, Llama 4 Maverick, and Mixtral-8x7B. It is the backbone of all three specialist agents.

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Property' 'frags': [ParaFrag(__tag__='para', bold=1, fontName='Helvetica-Bold', fontSize=8, greek=0, italic=0, link=[], rise=0, text='Property', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Detail' 'frags': [ParaFrag(__tag__='para', bold=1, fontName='Helvetica-Bold', fontSize=8, greek=0, italic=0, link=[], rise=0, text='Detail', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph
Model	Llama 4 Scout (best free) / Mixtral-8x7B (backup)
Speed	700–1,000 tokens/second — fast enough for real-time agent streaming
Free limits	14,400 requests/day — covers 200+ daily analysis runs comfortably

Context	128K tokens — handles full earnings reports + indicator data in one call
Endpoint	OpenAI-compatible: https://api.groq.com/openai/v1
Tool calling	Full function calling support — LangGraph agent tools work natively
Sign-up	console.groq.com — API key in 2 minutes, no credit card

2.2 Secondary LLM: Google Gemini 2.0 Flash (Free Tier)

Google AI Studio provides Gemini 2.0 Flash free of charge. Used for the Orchestrator's final synthesis pass when the combined output from all agents is large, and for any task requiring long-context reasoning (10-K documents, multi-stock portfolio analysis).

Free quota	1,500 requests/day, 1 million tokens/minute
Context	1,000,000 token context window — entire annual reports in one call
Use case	Orchestrator synthesis, portfolio-wide analysis, PDF/filing ingestion
Endpoint	OpenAI-compatible via google-generativeai SDK
Sign-up	aistudio.google.com — free, no credit card

2.3 FinBERT Sentiment — CPU-Only

FinBERT (ProsusAI/finbert, 110M parameters) runs comfortably on CPU. Inference on a headline + 200 tokens takes ~0.3–0.8 seconds per article. Since sentiment is computed in background Celery tasks (not on the critical user-request path), CPU speed is entirely acceptable. No GPU needed.

> ■ ML Model Training (LSTM): Training the LSTM model does require a GPU. Use Kaggle Notebooks (free T4 GPU, 30 hrs/week) for the one-time training. Save the trained model weights to disk. The FastAPI backend loads and serves them on CPU — inference on CPU for a small LSTM is fast (< 1 second per prediction).

3. User Interaction Model

The application exposes seven primary user interaction patterns. Each maps to a specific backend flow and set of agents.

3.1 Single Stock Analysis

The core flow. User searches a ticker or company name. The app triggers all three agents (Fundamental, Technical, Sentiment) in parallel, then the Orchestrator synthesises a prediction. The result is streamed step-by-step via WebSocket.

- → User types 'Reliance' or 'RELIANCE.NS' in the search bar.
- → Ticker resolver maps input to canonical NSE symbol.
- → WebSocket connection opens to /ws/analyze/{symbol}.
- → Orchestrator invokes Fundamental, Technical, Sentiment agents in parallel.
- → Each agent streams its intermediate steps to the client as it works.
- → Prediction Agent receives all three outputs and generates final verdict.
- → UI displays: direction (Bullish/Bearish/Neutral), target price range, confidence %, rationale, key signals.
- → Analysis is saved to PostgreSQL and embedded into ChromaDB for future RAG retrieval.

3.2 Portfolio Analysis

User adds stocks to their portfolio with quantity and average buy price. The Portfolio Agent then analyses the full holdings: risk metrics, sector concentration, correlation, and rebalancing suggestions.

- → User navigates to Portfolio screen.
- → Adds holdings: ticker, quantity, avg buy price, date.
- → Taps 'Analyse Portfolio' — triggers Portfolio Agent.
- → System fetches current prices for all holdings via Finnhub.
- → Portfolio Agent computes: unrealised P&L;, Sharpe ratio, max drawdown, sector heatmap, correlation matrix.
- → Generates rebalancing suggestions using Modern Portfolio Theory.
- → Runs individual stock analysis for top 3 holdings by weight.
- → Returns a portfolio health score (0–100) with drill-down per holding.

3.3 Natural Language Query (Chat Mode)

User types a free-text question in a chat interface. The Orchestrator interprets intent, routes to appropriate agents, and replies conversationally. This is the 'AI advisor' UX.

- → User types: 'Is Infosys a good buy right now given high US rates?'
- → Orchestrator parses intent: single-stock analysis + macro context.
- → Routes to Technical Agent, Fundamental Agent, Sentiment Agent.
- → Macro context (US rate environment) retrieved from ChromaDB / Alpha Vantage economics endpoint.
- → Orchestrator synthesises a conversational response citing specific data points.
- → User can follow up: 'Compare it with Wipro' — session memory maintains context.
- → Each message pair stored in conversation history for RAG retrieval in future sessions.

3.4 Watchlist & Screener

User maintains a watchlist of stocks. The system runs background analysis on all watchlist stocks nightly and surfaces the top opportunities each morning.

- → User adds stocks to watchlist from the search screen.
- → Nightly Celery task (runs at 18:45 IST after market close) runs analysis on all watchlist stocks.
- → Results stored in PostgreSQL with a freshness timestamp.
- → Next morning, user opens app to see pre-computed 'Top Picks' ranked by signal strength.
- → Custom screener: user sets filters (RSI < 40, P/E < 20, positive sentiment) to filter universe.
- → Screener runs against pre-computed indicator table — no live API calls needed.
- → Results sorted by composite score: technical signal strength + fundamental score + sentiment.

3.5 Price & Signal Alerts

User sets conditions that trigger push notifications. The alert engine runs every 5 minutes during market hours via a Celery worker.

- → User defines alert: 'Notify me when HDFC Bank RSI drops below 35' or 'Alert when price > 1800'.
- → Alert stored in PostgreSQL with condition type, operator, threshold, ticker.
- → Alert Celery worker runs every 5 minutes: fetches latest quote + computed RSI from cache.
- → If condition met: sends push notification via Expo Notifications (mobile) or OS notification (desktop).
- → User can set: price above/below, RSI threshold, MACD crossover, sentiment score drop, earnings date reminder.
- → Alert history logged per user for review.

3.6 Historical Analysis Review

User can browse past analyses stored in the app. Useful for tracking prediction accuracy and reviewing the reasoning behind past decisions.

- → User opens 'Analysis History' from the sidebar.
- → Fetches analysis records from PostgreSQL, ordered by date.
- → Each record shows: ticker, date, prediction made, actual outcome (computed from subsequent price data).
- → Accuracy tracker: system computes % of past predictions that were correct within target range.
- → User can tap any past analysis to see full agent reasoning trace.
- → Export to CSV available for personal tracking.

3.7 Sector & Market Overview

A dashboard view showing the health of the overall market and sector-by-sector breakdown. No user-specific portfolio data — a macro view.

- → Dashboard auto-loads on app open.
- → Fetches Nifty 50, Nifty Bank, Nifty IT, Nifty Pharma sector index data via yfinance.
- → Displays: day change %, 52-week high/low, RSI for each index.
- → News sentiment aggregate: 'Market mood today' based on top 50 most-read financial news articles.
- → FII/DII flow data from NSE website (daily scrape).
- → Top gainers and losers from Nifty 500 universe.

4. Backend Communication Design

The backend exposes two communication interfaces: REST (HTTP) for CRUD and one-shot queries, and WebSocket for streaming agent analysis. Both run on the same FastAPI server.

4.1 REST API Endpoints

Method	Endpoint	Description	Auth
POST	/auth/register	Register new user	Public
POST	/auth/login	JWT login, returns access+refresh tokens	Public
POST	/auth/refresh	Refresh access token	Refresh token
GET	/stocks/search?q={query}	Search ticker by name or symbol	Bearer JWT
GET	/stocks/{symbol}/quote	Latest real-time quote (Redis cached 60s)	Bearer JWT
GET	/stocks/{symbol}/history	OHLCV history with timeframe param	Bearer JWT
GET	/stocks/{symbol}/analysis/latest	Last cached analysis for ticker	Bearer JWT
GET	/portfolio	Fetch user's portfolio holdings	Bearer JWT
POST	/portfolio	Add holding to portfolio	Bearer JWT
DELETE	/portfolio/{id}	Remove holding from portfolio	Bearer JWT
GET	/watchlist	Fetch user's watchlist	Bearer JWT
POST	/watchlist	Add stock to watchlist	Bearer JWT
GET	/alerts	List user's configured alerts	Bearer JWT
POST	/alerts	Create new price/indicator alert	Bearer JWT
DELETE	/alerts/{id}	Delete alert	Bearer JWT
GET	/analysis/history	Paginated past analyses for user	Bearer JWT
GET	/market/overview	Nifty indices + sector data	Bearer JWT
GET	/screener	Run screener with filter params	Bearer JWT

4.2 WebSocket Streaming Protocol

The WebSocket connection at /ws/analyze/{symbol} streams the agent reasoning process to the client in real time. Each message is a JSON object with a type field.

Message type	Payload	When sent
`analysis_start`	`{symbol, agents_invoked, timestamp}`	Immediately on connection
`agent_thinking`	`{agent, step, message}`	Each agent intermediate step
`agent_complete`	`{agent, output_json}`	When an agent finishes
`prediction_start`	`{message: 'Synthesising...'}`	Prediction agent starts

`prediction_complete`	`{direction, target_range, confidence, rationale, key_signals}`	Final verdict
`analysis_saved`	`{analysis_id}`	After DB write completes
`error`	`{code, message}`	On any agent failure

4.3 Client-side WebSocket Handling (React Native)

React Native WebSocket client — analysis flow

```
const ws = new WebSocket(`wss://api.stocksage.ai/ws/analyze/${symbol}`);
ws.onmessage = (event) => {
const msg = JSON.parse(event.data);
switch(msg.type) {
case 'agent_thinking':
dispatch(addThinkingStep({ agent: msg.agent, text: msg.message }));
break;
case 'agent_complete':
dispatch(setAgentOutput({ agent: msg.agent, data: msg.output_json }));
break;
case 'prediction_complete':
dispatch(setPrediction(msg));
ws.close();
break;
case 'error':
dispatch(setError(msg.message));
ws.close();
break;
}
};
ws.onerror = (e) => { /* fallback to REST polling */ };
```

5. Complete Free Tech Stack

Layer	Tool / Library	Version	Free Tier / Limit	Purpose
LLM – Primary	Groq API (Llama 4 Scout)	latest	14,400 req/day	All 3 specialist agents
LLM – Secondary	Google Gemini 2.0 Flash	latest	1,500 req/day, 1M ctx	Orchestrator synthesis, long docs
Agent Orchestration	LangGraph	0.2+	Open-source, unlimited	Multi-agent DAG with state
Agent Framework	LangChain	0.3+	Open-source, unlimited	Tools, prompts, memory
Market Data	yfinance	0.2.x	Unlimited (unofficial)	OHLCV, fundamentals, actions
Market Data 2	Alpha Vantage	free key	25 req/day	Tech indicators, news sentiment
Market Data 3	Finnhub	free key	60 req/min	Real-time quotes, earnings calendar
News	NewsAPI.org	free key	100 req/day	Financial news by ticker
News 2	GNews API	free key	100 req/day	Google News, India coverage
Sentiment NLP	FinBERT (HuggingFace)	110M	Local CPU, unlimited	Article sentiment scoring
Tech Analysis	pandas-ta	0.3.x	Open-source, unlimited	130+ indicators on DataFrames
Tech Analysis 2	TA-Lib	0.4.x	Open-source, unlimited	200+ indicators + candlestick patterns
ML – Tabular	XGBoost + scikit-learn	latest	Open-source, unlimited	Feature-based prediction ensemble
ML – Time Series	Prophet (Meta)	1.1.x	Open-source, unlimited	Trend + seasonality forecasting
ML – Sequence	PyTorch (LSTM)	2.x	Open-source; train on Kaggle GPU	Sequence model, CPU inference
Backend	FastAPI + Uvicorn	0.110+	Open-source, unlimited	REST + WebSocket API server
Task Queue	Celery + Redis	5.x	Open-source; Upstash free 10K/day	Async jobs, scheduled tasks
Primary DB	PostgreSQL (Supabase free)	15	500MB, 50K rows	Users, portfolios, analyses
Local DB	SQLite	built-in	Unlimited	Desktop app local storage
Vector DB	ChromaDB	0.5+	Open-source, local, unlimited	RAG embeddings, semantic search
Cache	Redis (local / Upstash)	7.x	Upstash free: 10K cmd/day	Quote cache, sessions, pub-sub
Embeddings	sentence-transformers	latest	Open-source, CPU, unlimited	Text embedding for ChromaDB
Mobile	React Native + Expo	SDK 51	Open-source, unlimited	iOS + Android from one codebase

Desktop	**Tauri v2**	**2.x**	**Open-source, unlimited**	**Windows/macOS desktop ~5MB binary**
Web/PWA	**Next.js 14**	**14**	**Open-source, unlimited**	**Browser fallback + shareable URLs**
Charting	**Lightweight Charts (TradingView)**	**4.x**	**Open-source, unlimited**	**Candlestick charts on all platforms**
Hosting	**Railway**	**—**	**$5 credit/month**	**FastAPI + Postgres deployment**
Observability	**LangSmith**	**—**	**Free developer tier**	**Agent tracing and debugging**
Training	**Kaggle Notebooks**	**—**	**30 GPU hrs/week (T4)**	**LSTM training, one-time + weekly**

6. Data Sources & Catalog

The full list of data types required, their sources, update frequency, and which agent consumes them.

Data Type	Fields	Free Source	Freq.	Agent
Daily OHLCV	open,high,low,close,volume,adj_close	yfinance	Nightly EOD	Technical
Intraday candles	5m/15m/1h OHLCV (2 months free)	yfinance	Market hours	Technical
Real-time quote	last_price,bid,ask,volume,change_%	Finnhub free tier	On request	Orchestrator
Options chain	CE/PE OI,IV,strike,expiry,Greeks	NSE website scrape	Daily	Technical
Income statement	Revenue,EBITDA,PAT,EPS — Q + Annual	yfinance .financials	Quarterly	Fundamental
Balance sheet	Assets,liabilities,equity,total_debt	yfinance .balance_sheet	Quarterly	Fundamental
Cash flow	Operating,investing,financing FCF	yfinance .cashflow	Quarterly	Fundamental
Financial ratios	P/E,P/B,EV/EBITDA,ROE,ROCE,D/E	yfinance .info + Screener	Daily	Fundamental
Promoter holding	promoter_%,pledge_%,FII_%,DII_%	Screener.in scrape	Weekly	Fundamental
Earnings calendar	result_date,EPS_est,EPS_actual	Finnhub calendar	Weekly	Fundamental
Corporate actions	dividends,splits,bonus_ratio,rights	yfinance .actions	Daily	Fundamental
Financial news	headline,body_200_tokens,source,pub_at	NewsAPI + GNews	30 min	Sentiment
Sentiment score	pos_prob,neg_prob,neutral_prob per art.	FinBERT (local CPU)	Post-fetch	Sentiment
Event tags	earnings,M&A;,mgmt,macro,regulatory	Groq/Llama 4 classifier	Post-fetch	Sentiment
Macro indicators	Repo rate,CPI,GDP,FII/DII flows	Alpha Vantage economics	Monthly	Fundamental
Sector indices	Nifty IT/Bank/Pharma OHLCV	yfinance (^CNXAUTO etc.)	Daily	Fundamental
Tech indicators	RSI,MACD,BB,ATR,EMA20/50/200,VWAP	pandas-ta (computed)	On OHLCV	Technical
Candlestick pats	Pattern name,bullish/bearish,confidence	TA-Lib (computed)	On OHLCV	Technical
S/R zones	Top 3 support + 3 resistance price levels	Volume profile (custom)	On OHLCV	Technical
User portfolio	ticker,qty,avg_buy_price,buy_date	User input → SQLite/PG	On user action	Portfolio
Past analyses	full_agent_outputs,ticker,date,verdict	PostgreSQL + ChromaDB	Post-run	RAG / All
User preferences	risk_appetite,watchlist,playbook_rules	PostgreSQL + ChromaDB	On user edit	Orchestrator

7. Data Ingestion Pipelines

Six pipelines must be built and scheduled. Each pipeline is an independent Celery task that can be triggered manually or on a cron schedule.

Pipeline 1 — Market Data Ingestion

Fetches and stores OHLCV data for all tracked tickers. Triggered nightly after market close and on-demand when a user first searches a ticker.

pipelines/market_data.py

```
import yfinance as yf
from celery import shared_task
from app.db import get_db
from app.cache import redis_client

@shared_task
def ingest_ohlcv(symbol: str, period: str = "5y"):
ticker = yf.Ticker(symbol)
df = ticker.history(period=period, interval="1d")
df = df.reset_index()
df.columns = [c.lower() for c in df.columns]
# Upsert into TimescaleDB ohlcv table
with get_db() as db:
for _, row in df.iterrows():
db.execute(
"""INSERT INTO ohlcv (symbol, date, open, high, low, close, volume)
VALUES (%s,%s,%s,%s,%s,%s,%s)
ON CONFLICT (symbol, date) DO UPDATE
SET close=EXCLUDED.close, volume=EXCLUDED.volume""",
(symbol, row['date'], row['open'], row['high'],
row['low'], row['close'], row['volume'])
)
# Cache latest quote in Redis (60s TTL)
latest = df.iloc[-1]
redis_client.setex(f"quote:{symbol}", 60, str(latest['close']))
return {"symbol": symbol, "rows": len(df)}

# Celery beat schedule — runs 18:30 IST on weekdays
CELERY_BEAT_SCHEDULE = {
"nightly-market-ingest": {
"task": "pipelines.market_data.ingest_all_tracked",
"schedule": crontab(hour=13, minute=0, day_of_week="1-5"), # 18:30 IST = 13:00 UTC
}
}
```

Pipeline 2 — Technical Feature Computation

Runs immediately after market data ingestion. Computes all technical indicators and candlestick patterns. Results stored as columns in the features table.

pipelines/technical_features.py

```
import pandas_ta as ta
import talib
import numpy as np

@shared_task
def compute_technical_features(symbol: str):
```

```
with get_db() as db:
rows = db.execute(
"SELECT date,open,high,low,close,volume FROM ohlcv WHERE symbol=%s ORDER BY date",
(symbol,)).fetchall()
df = pd.DataFrame(rows, columns=["date","open","high","low","close","volume"])

# Indicators via pandas-ta
df.ta.rsi(length=14, append=True) # RSI_14
df.ta.macd(append=True) # MACD_12_26_9, MACDh, MACDs
df.ta.bbands(length=20, append=True) # BBL, BBM, BBU, BBB, BBP
df.ta.atr(length=14, append=True) # ATRr_14
df.ta.ema(length=20, append=True) # EMA_20
df.ta.ema(length=50, append=True) # EMA_50
df.ta.ema(length=200, append=True) # EMA_200
df.ta.vwap(append=True) # VWAP_D
df.ta.stoch(append=True) # STOCHk_14_3_3, STOCHd

# Candlestick patterns via TA-Lib
o,h,l,c = df.open.values, df.high.values, df.low.values, df.close.values
patterns = {
"CDL_ENGULFING": talib.CDLENGULFING(o,h,l,c),
"CDL_HAMMER": talib.CDLHAMMER(o,h,l,c),
"CDL_DOJI": talib.CDLDOJI(o,h,l,c),
"CDL_MORNINGSTAR":talib.CDLMORNINGSTAR(o,h,l,c),
"CDL_SHOOTINGSTAR":talib.CDLSHOOTINGSTAR(o,h,l,c),
"CDL_HARAMI": talib.CDLHARAMI(o,h,l,c),
}
for pat_name, pat_values in patterns.items():
df[pat_name] = pat_values # 100=bullish, -100=bearish, 0=none

# Upsert features into DB
latest = df.iloc[-1].to_dict()
with get_db() as db:
db.execute(
"""INSERT INTO technical_features (symbol, date, rsi, macd, bb_upper, bb_lower,
atr, ema20, ema50, ema200, patterns_json)
VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s)
ON CONFLICT (symbol, date) DO UPDATE SET rsi=EXCLUDED.rsi, ...""",
(symbol, latest['date'], latest.get('RSI_14'), latest.get('MACD_12_26_9'),
latest.get('BBU_20_2.0'), latest.get('BBL_20_2.0'),
latest.get('ATRr_14'), latest.get('EMA_20'), latest.get('EMA_50'),
latest.get('EMA_200'), json.dumps({k:v for k,v in latest.items() if k.startswith('CDL_')}))
)
```

Pipeline 3 — News & Sentiment Pipeline

Fetches news every 30 minutes during market hours. Runs FinBERT locally on CPU. Classifies events with Groq/Llama 4. Stores composite sentiment score per ticker per day.

pipelines/sentiment.py

```
from transformers import pipeline as hf_pipeline
from newsapi import NewsApiClient
import hashlib

# Load FinBERT once at startup (CPU-friendly 110M model)
finbert = hf_pipeline(
"sentiment-analysis",
model="ProsusAI/finbert",
device=-1 # -1 = CPU
```

```
)
@shared_task
def run_sentiment_pipeline(symbol: str, company_name: str):
newsapi = NewsApiClient(api_key=NEWSAPI_KEY)
articles = newsapi.get_everything(
q=f"{company_name} OR {symbol}",
language="en",
sort_by="publishedAt",
from_param=(datetime.now() - timedelta(days=3)).strftime("%Y-%m-%d"),
page_size=20
).get("articles", [])
results = []
for art in articles:
# Deduplicate by headline hash
art_hash = hashlib.md5(art["title"].encode()).hexdigest()
if cache_exists(art_hash): continue
text = art["title"] + " " + (art.get("description") or "")[:200]
sentiment = finbert(text[:512])[0] # FinBERT max 512 tokens
score = sentiment["score"] if sentiment["label"] == "positive" else (
-sentiment["score"] if sentiment["label"] == "negative" else 0)
# Event classification via Groq (batch to save API calls)
results.append({"hash": art_hash, "title": art["title"], "score": score,
"pub_at": art["publishedAt"]})
cache_set(art_hash, ttl=86400)
if not results: return
# Weighted composite score: recent = 3x, last 24h = 2x, older = 1x
now = datetime.utcnow()
weighted_scores, weights = [], []
for r in results:
age_h = (now - parse_dt(r["pub_at"])).total_seconds() / 3600
w = 3 if age_h < 6 else (2 if age_h < 24 else 1)
weighted_scores.append(r["score"] * w)
weights.append(w)
composite = sum(weighted_scores) / sum(weights) if weights else 0
save_sentiment(symbol, composite, results)
```

Pipeline 4 — Fundamentals Ingestion

pipelines/fundamentals.py

```
@shared_task
def ingest_fundamentals(symbol: str):
ticker = yf.Ticker(symbol)
info = ticker.info # P/E, P/B, market_cap, 52w high/low etc.
# Quarterly financials
income_q = ticker.quarterly_financials
balance_q = ticker.quarterly_balance_sheet
cashflow_q= ticker.quarterly_cashflow
# Compute key ratios from raw financials
ratios = compute_ratios(info, income_q, balance_q, cashflow_q)
# e.g.: ROCE = EBIT / (Total Assets - Current Liabilities)
# D/E = Total Debt / Shareholder Equity
# FCF = Operating Cash Flow - CapEx
# DCF: simple 5-year projection
fcf = cashflow_q.loc["Free Cash Flow"].mean() if "Free Cash Flow" in cashflow_q.index else None
```

```
wacc = 0.10 # default 10% — can be user-customised
dcf_value = dcf_model(fcf, growth=0.12, terminal_g=0.05, wacc=wacc, years=5)

with get_db() as db:
db.execute(
"""INSERT INTO fundamentals (symbol, date, pe_ratio, pb_ratio, ev_ebitda,
roe, roce, debt_equity, fcf, dcf_intrinsic, market_cap, ratios_json)
VALUES (%s,NOW(),%s,%s,%s,%s,%s,%s,%s,%s,%s)
ON CONFLICT (symbol) DO UPDATE SET pe_ratio=EXCLUDED.pe_ratio, ...""",
(symbol, info.get("trailingPE"), info.get("priceToBook"),
info.get("enterpriseToEbitda"), info.get("returnOnEquity"),
ratios.get("roce"), ratios.get("debt_equity"), fcf, dcf_value,
info.get("marketCap"), json.dumps(ratios))
)
```

Pipeline 5 — ML Feature Store & Model Training

pipelines/ml_training.py (run on Kaggle GPU — one-time + weekly)

```
# Feature engineering: 40+ features per stock per day
def build_feature_matrix(symbol: str) -> pd.DataFrame:
ohlcv = load_ohlcv(symbol) # From PostgreSQL
tech = load_tech_features(symbol) # RSI, MACD, BB, ATR, etc.
fund = load_fundamentals(symbol) # P/E, ROCE, D/E
sent = load_sentiment(symbol) # Daily composite sentiment score

df = ohlcv.merge(tech, on="date").merge(fund, on="date", how="left")
.merge(sent, on="date", how="left")
df["sentiment"].fillna(0, inplace=True)

# Target: next 7-day return > 2% = 1, < -2% = -1, else = 0
df["target"] = np.where(df["close"].shift(-7) > df["close"]*1.02, 1,
np.where(df["close"].shift(-7) < df["close"]*0.98, -1, 0))
return df.dropna()

# Walk-forward training on Nifty 200 universe
from xgboost import XGBClassifier
import joblib

features = ["RSI_14","MACD_12_26_9","BBP_20_2.0","ATRr_14",
"EMA_20","EMA_50","volume_ratio","sentiment","pe_ratio","roce"]
X, y = df[features], df["target"]
model = XGBClassifier(n_estimators=300, max_depth=5, learning_rate=0.05,
use_label_encoder=False, eval_metric="mlogloss")
model.fit(X_train, y_train)
joblib.dump(model, "models/xgb_model.joblib")

# Prophet per-ticker (runs on CPU fine)
from prophet import Prophet
prophet_df = df[["date","close"]].rename(columns={"date":"ds","close":"y"})
m = Prophet(holidays=get_india_holidays())
m.fit(prophet_df)
future = m.make_future_dataframe(periods=30)
forecast = m.predict(future) # Returns trend + yhat_lower/upper
```

Pipeline 6 — Agent Run Pipeline (On-Demand)

agents/orchestrator.py — LangGraph StateGraph

```
from langgraph.graph import StateGraph, END
from typing import TypedDict, Annotated
import asyncio
```

```
class AnalysisState(TypedDict):
symbol: str
fundamental_output: dict
technical_output: dict
sentiment_output: dict
ml_scores: dict
prediction: dict
messages: list # streamed to WebSocket

def build_analysis_graph():
graph = StateGraph(AnalysisState)

# Add nodes
graph.add_node("fundamental_agent", run_fundamental_agent)
graph.add_node("technical_agent", run_technical_agent)
graph.add_node("sentiment_agent", run_sentiment_agent)
graph.add_node("ml_scores", run_ml_prediction)
graph.add_node("orchestrator", run_orchestrator_synthesis)

# Parallel execution: all three agents run concurrently from START
graph.set_entry_point("fundamental_agent") # LangGraph runs parallel via asyncio
graph.add_edge("fundamental_agent", "orchestrator")
graph.add_edge("technical_agent", "orchestrator")
graph.add_edge("sentiment_agent", "orchestrator")
graph.add_edge("ml_scores", "orchestrator")
graph.add_edge("orchestrator", END)

return graph.compile()

# WebSocket handler
async def stream_analysis(symbol: str, websocket: WebSocket):
graph = build_analysis_graph()
state = {"symbol": symbol, "messages": []}
async for event in graph.astream(state):
await websocket.send_json({"type": "agent_thinking", **event})
final = event # last event = prediction_complete
await websocket.send_json({"type": "prediction_complete", **final["prediction"]})
```

8. Agent Design

8.1 Fundamental Analyst Agent

Evaluates intrinsic value by reading financial statements and ratios. Uses Groq/Llama 4. Calls three tools: get_financial_ratios, get_dcf_estimate, get_peer_comparison.

agents/fundamental_agent.py

```
SYSTEM_PROMPT = """You are a CFA-level fundamental analyst specialising in Indian equities.
You receive structured JSON data: financial ratios, DCF estimate, and peer comparison.
Your job:
1. Assess whether the stock is UNDERVALUED, FAIRLY_VALUED, or OVERVALUED with reasoning.
2. Flag any red flags: rising debt, falling promoter holding, negative FCF, margin compression.
3. Compare against sector peers. Is this stock best-in-class, average, or lagging?
4. Return ONLY a valid JSON object with keys:
valuation_verdict, dcf_margin_of_safety_pct, red_flags (list), peer_rank (1=best),
key_ratios_summary, overall_score (0-10).
No markdown. No preamble."""

async def run_fundamental_agent(state: AnalysisState) -> AnalysisState:
symbol = state["symbol"]

# Load pre-computed data (fast — no API call on critical path)
ratios = db.get_fundamentals(symbol)
dcf = db.get_dcf(symbol)
peers = db.get_peers(symbol, n=3)

payload = json.dumps({"ratios": ratios, "dcf": dcf, "peers": peers})

client = Groq(api_key=GROQ_API_KEY)
response = client.chat.completions.create(
model="llama-4-scout-17b-16e-instruct",
messages=[
{"role": "system", "content": SYSTEM_PROMPT},
{"role": "user", "content": f"Analyse {symbol}: {payload}"}
],
response_format={"type": "json_object"},
max_tokens=800
)
output = json.loads(response.choices[0].message.content)
state["fundamental_output"] = output
return state
```

8.2 Technical Analyst Agent

Reads pre-computed indicators and patterns from the features table. Analyses multi-timeframe alignment and identifies key S/R levels.

agents/technical_agent.py

```
SYSTEM_PROMPT = """You are a seasoned technical analyst. You receive indicator data for a stock
across three timeframes: 15m, 1H, and 1D. Your job:
1. Identify the dominant TREND: UPTREND, DOWNTREND, or SIDEWAYS with reasoning.
2. Assess MOMENTUM using RSI, MACD, Stochastic.
3. Report any candlestick PATTERNS detected (bullish or bearish, strength).
4. Assess TIMEFRAME ALIGNMENT: are all 3 timeframes in agreement?
5. List the 3 most important SUPPORT and 3 RESISTANCE levels.
6. Give a final SIGNAL: STRONG_BUY / BUY / NEUTRAL / SELL / STRONG_SELL.
Return ONLY valid JSON. No markdown."""

async def run_technical_agent(state: AnalysisState) -> AnalysisState:
```

```
symbol = state["symbol"]
tech_features = {
"1D": db.get_tech_features(symbol, "1d"),
"1H": db.get_tech_features(symbol, "1h"),
"15m": db.get_tech_features(symbol, "15m"),
}
sr_zones = compute_sr_zones(symbol) # Volume profile S/R
payload = json.dumps({"indicators": tech_features, "sr_zones": sr_zones})
client = Groq(api_key=GROQ_API_KEY)
response = client.chat.completions.create(
model="llama-4-scout-17b-16e-instruct",
messages=[
{"role": "system", "content": SYSTEM_PROMPT},
{"role": "user", "content": f"Analyse {symbol}: {payload}"}
],
response_format={"type": "json_object"},
max_tokens=800
)
state["technical_output"] = json.loads(response.choices[0].message.content)
return state
```

8.3 Sentiment Analyst Agent

Reads pre-scored news from the sentiment table (FinBERT already ran in the background). Uses Groq to interpret the sentiment context and classify high-impact events.

agents/sentiment_agent.py

```
SYSTEM_PROMPT = """You are a financial news analyst. You receive:
1. A composite sentiment score (-1 to +1) for the stock.
2. The top 5 most recent news headlines with individual scores.
3. Any detected high-impact event flags.
Your job:
1. Summarise the current market MOOD for this stock.
2. Identify if any HIGH_IMPACT events could override technical signals.
Categories: EARNINGS_SURPRISE, MA_MERGER, MANAGEMENT_CHANGE, REGULATORY, MACRO.
3. Estimate the SENTIMENT_TREND: improving, stable, or deteriorating vs. 7 days ago.
4. Give a SENTIMENT_SIGNAL: VERY_POSITIVE / POSITIVE / NEUTRAL / NEGATIVE / VERY_NEGATIVE.
Return ONLY valid JSON."""
async def run_sentiment_agent(state: AnalysisState) -> AnalysisState:
symbol = state["symbol"]
sent_data = db.get_sentiment(symbol) # Pre-computed by background pipeline
client = Groq(api_key=GROQ_API_KEY)
response = client.chat.completions.create(
model="llama-4-scout-17b-16e-instruct",
messages=[
{"role": "system", "content": SYSTEM_PROMPT},
{"role": "user", "content": json.dumps(sent_data)}
],
response_format={"type": "json_object"},
max_tokens=600
)
state["sentiment_output"] = json.loads(response.choices[0].message.content)
return state
```

8.4 Orchestrator Agent (Synthesis + Prediction)

Receives outputs from all three agents plus ML model scores. Uses Google Gemini (free tier) for the final synthesis pass — its larger context handles the full combined output. Produces the user-facing prediction.

agents/orchestrator_synthesis.py

```
SYSTEM_PROMPT = """You are the lead investment analyst. You receive structured outputs from
three specialist agents (Fundamental, Technical, Sentiment) and ML model probability scores.
Your job: synthesise all signals into a single, actionable investment view.

Rules:
- If fundamental says OVERVALUED + technical says STRONG_SELL + sentiment is NEGATIVE = STRONG SELL.
- A high-impact event (e.g. earnings miss) OVERRIDES technical signals.
- ML confidence < 0.5 = reduce conviction of final signal.
- Always cite which agent drove the final conclusion.

Return ONLY valid JSON with keys:
direction (BULLISH/BEARISH/NEUTRAL),
conviction (HIGH/MEDIUM/LOW),
confidence_pct (0-100),
target_price_range [low, high],
horizon_days (7, 14, or 30),
rationale (3-4 sentences, plain English for retail investor),
key_signals (list of 3-5 bullet strings),
risk_factors (list of 2-3 strings),
do_not_trade_if (single string warning condition)."""

async def run_orchestrator_synthesis(state: AnalysisState) -> AnalysisState:
payload = {
"fundamental": state["fundamental_output"],
"technical": state["technical_output"],
"sentiment": state["sentiment_output"],
"ml_scores": state["ml_scores"],
}

# Use Gemini for synthesis (larger context, free tier)
import google.generativeai as genai
genai.configure(api_key=GEMINI_API_KEY)
model = genai.GenerativeModel("gemini-2.0-flash")
response = model.generate_content(
SYSTEM_PROMPT + "\n\nAgent outputs:\n" + json.dumps(payload),
generation_config={"response_mime_type": "application/json"}
)
state["prediction"] = json.loads(response.text)
return state
```

9. Frontend Architecture

9.1 Mobile (React Native + Expo)

Five main screens, all sharing the same FastAPI backend via a unified API client.

Home / Dashboard: Market overview: Nifty indices, sector heatmap, top movers, FII/DII, market mood score.

Stock Search & Analysis: Search bar → results list → stock detail screen with candlestick chart (Lightweight Charts), agent reasoning stream, prediction card.

Portfolio: Holdings list with P&L;, portfolio health score, sector pie chart, correlation matrix, rebalancing suggestions.

Watchlist & Screener: Watchlist with overnight analysis results, custom screener with filter chips (RSI, P/E, sentiment).

Alerts & Settings: Create/manage price and indicator alerts, notification preferences, user preferences / playbook rules.

9.2 State Management

State management pattern (Zustand + React Query)

```
// Global client state: Zustand
const useAnalysisStore = create((set) => ({
currentSymbol: null,
streamingSteps: [], // Live agent thinking steps
agentOutputs: {}, // Keyed by agent name
prediction: null,
addStep: (step) => set((s) => ({streamingSteps: [...s.streamingSteps, step]})),
setPrediction: (p) => set({prediction: p}),
}));
// Server state: React Query (REST endpoints)
const { data: portfolio } = useQuery(['portfolio'], fetchPortfolio, {
staleTime: 5 * 60 * 1000, // 5 minutes
});
// WebSocket hook
function useAnalysisStream(symbol) {
const store = useAnalysisStore();
useEffect(() => {
if (!symbol) return;
const ws = new WebSocket(`${WS_BASE}/ws/analyze/${symbol}`);
ws.onmessage = ({data}) => {
const msg = JSON.parse(data);
if (msg.type === 'agent_thinking') store.addStep(msg);
if (msg.type === 'prediction_complete') store.setPrediction(msg);
};
return () => ws.close();
}, [symbol]);
}
```

9.3 Desktop (Tauri v2 + React)

The Tauri app is the same React codebase bundled with a Rust shell. Key additions: system tray icon with quick-access analysis, OS-level push notifications for alerts, SQLite local storage via Tauri's SQL plugin (allows offline access to last-fetched data), and a local Celery-like scheduler via Tauri's cron plugin for offline indicator

computation.

10. Project File Structure

Full monorepo structure

```
stocksage/
■■■ backend/ # FastAPI + LangGraph
■ ■■■ app/
■ ■ ■■■ main.py # FastAPI app, route registration
■ ■ ■■■ config.py # Env vars: GROQ_KEY, GEMINI_KEY, DB_URL
■ ■ ■■■ auth/
■ ■ ■ ■■■ router.py # /auth/* endpoints
■ ■ ■ ■■■ jwt.py # JWT encode/decode helpers
■ ■ ■■■ api/
■ ■ ■ ■■■ stocks.py # /stocks/* REST endpoints
■ ■ ■ ■■■ portfolio.py # /portfolio/* endpoints
■ ■ ■ ■■■ watchlist.py # /watchlist/* endpoints
■ ■ ■ ■■■ alerts.py # /alerts/* endpoints
■ ■ ■ ■■■ market.py # /market/* endpoints
■ ■ ■ ■■■ screener.py # /screener endpoint
■ ■ ■ ■■■ ws.py # WebSocket /ws/analyze/{symbol}
■ ■ ■■■ agents/
■ ■ ■ ■■■ orchestrator.py # LangGraph StateGraph builder
■ ■ ■ ■■■ fundamental.py # Fundamental agent (Groq)
■ ■ ■ ■■■ technical.py # Technical agent (Groq)
■ ■ ■ ■■■ sentiment.py # Sentiment agent (Groq)
■ ■ ■ ■■■ prediction.py # Orchestrator synthesis (Gemini)
■ ■ ■■■ pipelines/
■ ■ ■ ■■■ market_data.py # yfinance OHLCV ingestion
■ ■ ■ ■■■ technical_features.py# pandas-ta + TA-Lib computation
■ ■ ■ ■■■ sentiment_pipeline.py# FinBERT + NewsAPI pipeline
■ ■ ■ ■■■ fundamentals.py # yfinance + Screener.in pipeline
■ ■ ■ ■■■ ml_inference.py # XGBoost + Prophet prediction
■ ■ ■■■ models/
■ ■ ■ ■■■ xgb_model.joblib # Trained XGBoost (update weekly)
■ ■ ■ ■■■ prophet_models/ # Per-ticker Prophet models
■ ■ ■ ■■■ lstm_weights.pt # Trained LSTM (PyTorch)
■ ■ ■■■ db/
■ ■ ■ ■■■ connection.py # SQLAlchemy async engine
■ ■ ■ ■■■ migrations/ # Alembic migration files
■ ■ ■ ■■■ schema.sql # Full PostgreSQL schema
■ ■ ■■■ cache/
■ ■ ■ ■■■ redis_client.py # Redis connection + helpers
■ ■ ■■■ rag/
■ ■ ■ ■■■ chroma_client.py # ChromaDB setup
■ ■ ■ ■■■ embeddings.py # sentence-transformers embedding
■ ■ ■■■ celery_app.py # Celery app + beat schedule
■ ■■■ requirements.txt # All Python dependencies
■ ■■■ Dockerfile # For Railway deployment
■ ■■■ .env.example # GROQ_KEY, GEMINI_KEY, DB_URL, ...
■
■■■ mobile/ # React Native + Expo
■ ■■■ app/ # Expo Router (file-based routing)
■ ■ ■■■ (tabs)/
■ ■ ■ ■■■ index.tsx # Home/Dashboard
■ ■ ■ ■■■ search.tsx # Stock search + analysis
■ ■ ■ ■■■ portfolio.tsx # Portfolio screen
```

```
■ ■ ■ ■■■ watchlist.tsx # Watchlist + screener
■ ■ ■ ■■■ alerts.tsx # Alerts + settings
■ ■ ■■■ stock/[symbol].tsx # Stock detail with stream
■ ■■■ components/
■ ■ ■■■ CandlestickChart.tsx # Lightweight Charts via WebView
■ ■ ■■■ AgentStream.tsx # Live agent reasoning display
■ ■ ■■■ PredictionCard.tsx # Final verdict card
■ ■ ■■■ PortfolioHeatmap.tsx # Sector concentration chart
■ ■ ■■■ AlertCard.tsx # Alert item component
■ ■■■ store/
■ ■ ■■■ analysisStore.ts # Zustand: streaming state
■ ■ ■■■ userStore.ts # Zustand: auth + preferences
■ ■■■ hooks/
■ ■ ■■■ useAnalysisStream.ts # WebSocket hook
■ ■ ■■■ useQuote.ts # React Query: live quote
■ ■■■ api/
■ ■ ■■■ client.ts # Axios instance + interceptors
■ ■■■ package.json
■
■■■ desktop/ # Tauri v2 + React
■ ■■■ src-tauri/
■ ■ ■■■ src/main.rs # Tauri entry point
■ ■ ■■■ tauri.conf.json # App config, permissions
■ ■ ■■■ Cargo.toml # Rust dependencies
■ ■■■ src/ # Same React components as mobile
■
■■■ ml_training/ # Run on Kaggle — not deployed
■ ■■■ train_xgboost.ipynb # XGBoost training notebook
■ ■■■ train_lstm.ipynb # LSTM training notebook
■ ■■■ train_prophet.py # Prophet per-ticker training
■
■■■ docs/
■■■ SPEC.pdf # This document
```

11. Environment Setup

11.1 Required API Keys (All Free)

Key name	Where to get	Set in .env as
Groq API	console.groq.com → API Keys	`GROQ_API_KEY`
Google Gemini	aistudio.google.com → Get API key	`GEMINI_API_KEY`
Alpha Vantage	alphavantage.co/support/#api-key	`ALPHA_VANTAGE_KEY`
Finnhub	finnhub.io/register	`FINNHUB_KEY`
NewsAPI	newsapi.org/register	`NEWSAPI_KEY`
GNews	gnews.io (free)	`GNEWS_KEY`
Upstash Redis	upstash.com (free tier)	`REDIS_URL`
Supabase	supabase.com (free tier)	`DATABASE_URL`

11.2 Backend Setup

Terminal — backend setup

```
# 1. Clone repo and create virtual environment
git clone https://github.com/your-org/stocksage && cd stocksage/backend
python -m venv venv && source venv/bin/activate # Windows: venv\Scripts\activate

# 2. Install dependencies
pip install fastapi uvicorn[standard] langchain langgraph langchain-groq \
google-generativeai yfinance pandas-ta TA-Lib finnhub newsapi-python \
transformers sentence-transformers chromadb celery redis \
sqlalchemy alembic psycopg2-binary prophet xgboost scikit-learn \
joblib torch beautifulsoup4 requests python-jose[cryptography]

# 3. Copy and fill environment file
cp .env.example .env # Fill in all API keys

# 4. Run database migrations
alembic upgrade head

# 5. Start Redis (local)
redis-server &

# 6. Start Celery worker (background pipelines)
celery -A app.celery_app worker --loglevel=info &

# 7. Run initial data ingestion for test tickers
python -m app.pipelines.market_data ingest RELIANCE.NS TCS.NS INFY.NS

# 8. Start FastAPI server
uvicorn app.main:app --reload --port 8000

# FinBERT model download (one-time, ~450MB)
python -c "from transformers import pipeline; pipeline('sentiment-analysis',
model='ProsusAI/finbert')"

# Verify endpoints
curl http://localhost:8000/stocks/search?q=reliance
curl http://localhost:8000/market/overview
```

12. Implementation Roadmap

Phase 1 Data Foundation — Weeks 1–2

- ✓ Set up PostgreSQL (Supabase free) and SQLite schemas
- ✓ Build Pipeline 1: yfinance OHLCV ingestion for Nifty 50 universe
- ✓ Build Pipeline 2: pandas-ta + TA-Lib technical feature computation
- ✓ Set up Redis caching layer
- ✓ Build /stocks/search, /stocks/{symbol}/quote, /stocks/{symbol}/history REST endpoints
- ✓ Verify: query any NSE ticker and get full OHLCV + indicators

Phase 2 Sentiment Pipeline + ChromaDB — Weeks 3–4

- ✓ Build Pipeline 3: NewsAPI + GNews fetcher with deduplication
- ✓ Integrate FinBERT (CPU): run on fetched articles, store composite score
- ✓ Set up ChromaDB with sentence-transformers embeddings
- ✓ Build /stocks/{symbol}/sentiment REST endpoint
- ✓ Test: run full news + sentiment pipeline for RELIANCE, TCS, HDFC

Phase 3 Fundamentals Pipeline + Screener — Weeks 5–6

- ✓ Build Pipeline 4: yfinance fundamentals + ratio calculator + DCF model
- ✓ Build Screener.in scraper for ROCE, promoter holding
- ✓ Implement Finnhub earnings calendar sync
- ✓ Build /stocks/{symbol}/fundamentals and /screener endpoints
- ✓ Test: full fundamental profile for 50 Nifty 200 stocks

Phase 4 Three Agents + LangGraph — Weeks 7–8

- ✓ Set up Groq API client and Gemini client
- ✓ Build Fundamental Agent with system prompt + JSON output schema
- ✓ Build Technical Agent with multi-timeframe indicator payload
- ✓ Build Sentiment Agent reading pre-computed scores
- ✓ Wire all three into LangGraph StateGraph with parallel execution
- ✓ Build WebSocket endpoint /ws/analyze/{symbol}
- ✓ Test: end-to-end stream for a single stock analysis

Phase 5 ML Models + Prediction Agent — Weeks 9–10

- ✓ Build feature store on Kaggle Notebook (GPU training session)
- ✓ Train XGBoost model on Nifty 200 5-year dataset
- ✓ Train Prophet per-ticker for top 50 stocks
- ✓ Save models: xgb_model.joblib, prophet_models/
- ✓ Build ML inference pipeline (CPU): load models, run prediction on request
- ✓ Build Prediction / Orchestrator Agent with Gemini synthesis
- ✓ Add Celery beat schedule for weekly model retraining

Phase 6 **Frontend + Deployment** Weeks 11–12

- ✓ Build React Native Expo app with 5 screens
- ✓ Integrate Lightweight Charts via WebView for candlesticks
- ✓ Build AgentStream component for live WebSocket reasoning display
- ✓ Build PredictionCard component for final verdict
- ✓ Build Tauri desktop app (wrap same React codebase)
- ✓ Deploy FastAPI + Celery to Railway (free tier)
- ✓ Set up Celery beat for nightly pipelines
- ✓ Build Alert engine: Celery worker checking conditions every 5 min
- ✓ End-to-end QA on mobile + desktop

13. Appendix: Database Schema

schema.sql — PostgreSQL (Supabase compatible)

```
-- Users
CREATE TABLE users (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
email TEXT UNIQUE NOT NULL,
password_hash TEXT NOT NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);
-- OHLCV (TimescaleDB hypertable for fast time-series queries)
CREATE TABLE ohlcv (
symbol TEXT NOT NULL,
date DATE NOT NULL,
open NUMERIC(12,4),
high NUMERIC(12,4),
low NUMERIC(12,4),
close NUMERIC(12,4),
volume BIGINT,
PRIMARY KEY (symbol, date)
);
-- SELECT create_hypertable('ohlcv', 'date'); -- if TimescaleDB available
-- Technical features (pre-computed indicators)
CREATE TABLE technical_features (
symbol TEXT NOT NULL,
date DATE NOT NULL,
timeframe TEXT NOT NULL DEFAULT '1d',
rsi NUMERIC(6,2),
macd NUMERIC(10,4),
macd_signal NUMERIC(10,4),
bb_upper NUMERIC(12,4),
bb_lower NUMERIC(12,4),
atr NUMERIC(10,4),
ema20 NUMERIC(12,4),
ema50 NUMERIC(12,4),
ema200 NUMERIC(12,4),
patterns_json JSONB, -- { CDL_ENGULFING: 100, CDL_DOJI: 0 }
sr_zones_json JSONB, -- { support: [x,y,z], resistance: [a,b,c] }
PRIMARY KEY (symbol, date, timeframe)
);
-- Fundamentals
CREATE TABLE fundamentals (
symbol TEXT PRIMARY KEY,
updated_at TIMESTAMPTZ DEFAULT NOW(),
pe_ratio NUMERIC(8,2),
pb_ratio NUMERIC(8,2),
ev_ebitda NUMERIC(8,2),
roe NUMERIC(8,4),
roce NUMERIC(8,4),
debt_equity NUMERIC(8,4),
fcf BIGINT,
dcf_intrinsic NUMERIC(12,2),
market_cap BIGINT,
promoter_pct NUMERIC(5,2),
pledge_pct NUMERIC(5,2),
```

```
ratios_json JSONB
);
-- Sentiment
CREATE TABLE sentiment (
symbol TEXT NOT NULL,
date DATE NOT NULL,
composite_score NUMERIC(5,4), -- -1 to +1
article_count INT,
top_news_json JSONB,
event_flags_json JSONB,
PRIMARY KEY (symbol, date)
);
-- Portfolios and holdings
CREATE TABLE portfolios (id UUID PRIMARY KEY, user_id UUID REFERENCES users(id), name TEXT);
CREATE TABLE holdings (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
portfolio_id UUID REFERENCES portfolios(id),
symbol TEXT NOT NULL,
quantity NUMERIC(12,4),
avg_buy_price NUMERIC(12,4),
buy_date DATE
);
-- Watchlist
CREATE TABLE watchlist (id UUID PRIMARY KEY, user_id UUID REFERENCES users(id), symbol TEXT);
-- Alerts
CREATE TABLE alerts (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
user_id UUID REFERENCES users(id),
symbol TEXT,
condition TEXT, -- 'price_above','rsi_below','macd_crossover'
threshold NUMERIC(12,4),
is_active BOOLEAN DEFAULT TRUE,
created_at TIMESTAMPTZ DEFAULT NOW()
);
-- Analysis history
CREATE TABLE analyses (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
user_id UUID REFERENCES users(id),
symbol TEXT NOT NULL,
created_at TIMESTAMPTZ DEFAULT NOW(),
fundamental_json JSONB,
technical_json JSONB,
sentiment_json JSONB,
prediction_json JSONB, -- final verdict
chroma_doc_id TEXT -- reference back to ChromaDB embedding
);
-- Indexes
CREATE INDEX idx_ohlcv_symbol ON ohlcv(symbol);
CREATE INDEX idx_tech_symbol_date ON technical_features(symbol, date DESC);
CREATE INDEX idx_analyses_user ON analyses(user_id, created_at DESC);
```

Key Requirements Summary

■ Total monthly cost: Rs 0 during development and early production. Scale point: Groq free tier (14,400 req/day) supports ~100-200 analysis runs/day. First paid upgrade needed: Groq Developer plan ($9/mo) when user base exceeds ~500 daily active users. Database: Supabase free tier (500MB) supports ~10,000 analyses before upgrade.

Document compiled by StockSage AI — Technical Architecture Team. For Claude Code: implement in the order defined in Section 12 (Roadmap). Begin with Phase 1 database setup and market data ingestion. All API keys must be in .env before running any agent code.